

Name: Rhilo Sotto  
RED ID: 130551574  
Date: 11/15/2024

# Lab 7 Report

## 1. What does your program do?

The program uses analog to digital conversions, timers, and interrupts to convert analog input signals ranging from 0V to the reference voltage to make an LED dim and brighten. The program runs continuously, and since there is no break statement, it will run indefinitely until power is lost.

## 2. What variables/registers does your program use?

- **dutyCycle** (global uint16\_t variable)
- **DDRB** (register)
- **DDRC** (register)
- **ADMUX** (register)
- **ADCSRA** (register)
- **ADCSRB** (register)
- **TCCR0A** (register)
- **OCR0A** (register)
- **TIMSK0** (register)
- **TCCR2A** (register)
- **OCR2A** (register)
- **OCR2B** (register)
- **TIMSK2** (register)
- **TCCR0B** (register)
- **TCCR2B** (register)
- **variable** (uint16\_t variable)
- **PORTB** (register)

### 3. What is the main algorithm/logic of your program?

The main program calls the **init** function to initialize the DDRB register to set the LED on PINB5 as an output.

Then it calls the **adc\_init** function to initialize the the ADC settings, setting DDRC pin 1 as an input, setting the MUX to ADC1 (PINC1), enabling ADC, auto trigger, the conversion complete interrupt, the prescaler to x128, and to auto trigger on timer-0's compare match A.

Then it calls the **timer\_init** function to set Timer 0's and 2's modes to CTC which will cause the clock to reset upon reaching the value stored in OCR0A and OCR2A. TCCR0B and TCCR2B are used to set the prescaler value to x1024 and x128 and start the timers, which were chosen since the required interrupt period given my ID is 5+1ms, and the PWM frequency is 500Hz (2ms) respectively. These can be achieved with max periods of 16.384 ms and 2.048ms in CTC mode and enabling the interrupts.

Global interrupts are then enabled in main.

The program enters the infinite while loop.

The program has four interrupt service routines.

1. **ISR (TIMER0\_COMPA\_vect)** is Timer0's compare register A interrupt, which occurs every 6ms, it starts ADC conversion.
2. **ISR (ADC\_vect)** is the ADC's conversion complete interrupt, which stores the ADC result in an uint16\_t variable, and the corresponding duty cycle is calculated, shifting the result and accounting for prescalers, and set to OCR2B.
3. **ISR (TIMER2\_COMPA\_vect)** is Timer2's compare register A interrupt, which turns on the LED, except if the variable button has a value of 0, which represents a 0% duty cycle of not turning the LED on at all.
4. **ISR (TIMER2\_COMPB\_vect)** is Timer2's compare register B interrupt, which turns off the LED.

### 4. What external hardware connections did you make?

The potentiometer's ground, Vref, and output are connected to the microcontroller's GND, AREF, and PINC1 pins respectively. The LED is onboard, wired to PB5.

## Appendix (Code)

```
/*
 * Lab7.c
 *
 * Created: 11/5/2024 1:30:00 PM
 * Author : rnsot
 */

#define F_CPU 16000000UL // 16MHz clock frequency

#include <avr/io.h>
#include <avr/interrupt.h>

// REDID: 130551574
// X = 5
// Timer interrupt period = (5 + 1) ms = 6ms
// Y = 7
// Z = 4
// PWM frequency = (4 + 1) * 100Hz = 500Hz
// 2 ms period

uint16_t dutyCycle = 0;

void init(void) {
    DDRB |= (1 << DDRB5); // output to LED
}

void adc_init(void) {
    DDRC &= ~(1 << 1); // Y = 7 % 6 = 1
    ADMUX |= (1 << REFS0) | (1 << MUX0); // VCC reference, ADC1 (PINC1)
    ADCSRA |= (1 << ADEN) | (1 << ADSC) | (1 << ADIFSC);
    // enable ADC, auto trigger, and conversion complete interrupt
    ADCSRA |= (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);
    // Prescaler x128, 125kHz frequency (between 50kHz and 200kHz)
    ADCSRB |= (1 << ADTS1) | (1 << ADTS0); // triggers on timer0 compare match A
}

void timer_init(void) {
    // F_CPU = 16MHz, want 6ms and 2ms
    // TIMER 0 - interrupt timer (6ms)
    // ps = 1024, NM -> 16384us = 16.384ms **

    TCCR0A |= (1 << WGM01); // CTC Mode
```

```
OCR0A = 93; // timer period (6ms/16.384ms * 256 time steps = 93.75 time steps - 1 = 93
OCRA)
```

```
TIMSK0 |= (1 << OCIE0A); // set ISR COMPA vector
```

```
// TIMER 2 - PWM timer (2ms)
// ps = 128, NM -> 2048us = 2.048ms **
```

```
TCCR2A |= (1 << WGM21); // CTC Mode
```

```
OCR2A = 249; // timer period (2ms/2.048ms * 256 time steps = 250 time steps - 1 = 249
OCRA)
```

```
TIMSK2 |= (1 << OCIE2A) | (1 << OCIE2B); // set ISR COMPA and COMPB vector
```

```
TCCR0B |= (1 << CS02) | (1 << CS00); // Pre-scaler set to 1024, STARTS timer0
```

```
TCCR2B |= (1 << CS22) | (1 << CS20); // Pre-scaler set to 128, STARTS timer2
```

```
}
```

```
int main(void) {
```

```
    init(); // initialize inputs and outputs
    adc_init(); // initialize ADC settings
    timer_init(); // timer settings
```

```
    sei(); // enable global interrupts
```

```
    while (1) {
    }
```

```
}
```

```
ISR (TIMER0_COMPA_vect) { // Timer0 compare register A interrupt
```

```
    // start ADC conversion
    ADCSRA |= (1 << ADSC);
```

```
}
```

```
ISR (ADC_vect) { // ADC auto trigger and conversion complete interrupt
```

```
    uint16_t variable = ADC;
    dutyCycle = (OCR2A * (variable >> 2)) >> 10; // shift 10 bit ADC result and divide by
```

```
1024
```

```
    OCR2B = dutyCycle; // adjusts LED brightness
```

```
}
```

```
ISR (TIMER2_COMPA_vect) { // Timer2 compare register A interrupt  
    if (OCR2B > 0) PORTB |= (1 << PORTB5); // HIGH, LED ON
```

```
}
```

```
ISR (TIMER2_COMPB_vect) { // Timer2 compare register B interrupt  
    PORTB &= ~(1 << PORTB5); // LOW, LED OFF
```

```
}
```